

MBA EN **INGENIERIA DE SOFTWARE**

Gestión de Pruebas (TDD) I-II

Prof. Helder Prado Santos

La responsabilidad por la idoneidad, originalidad y licitud de los contenidos didácticos presentados es del profesor.

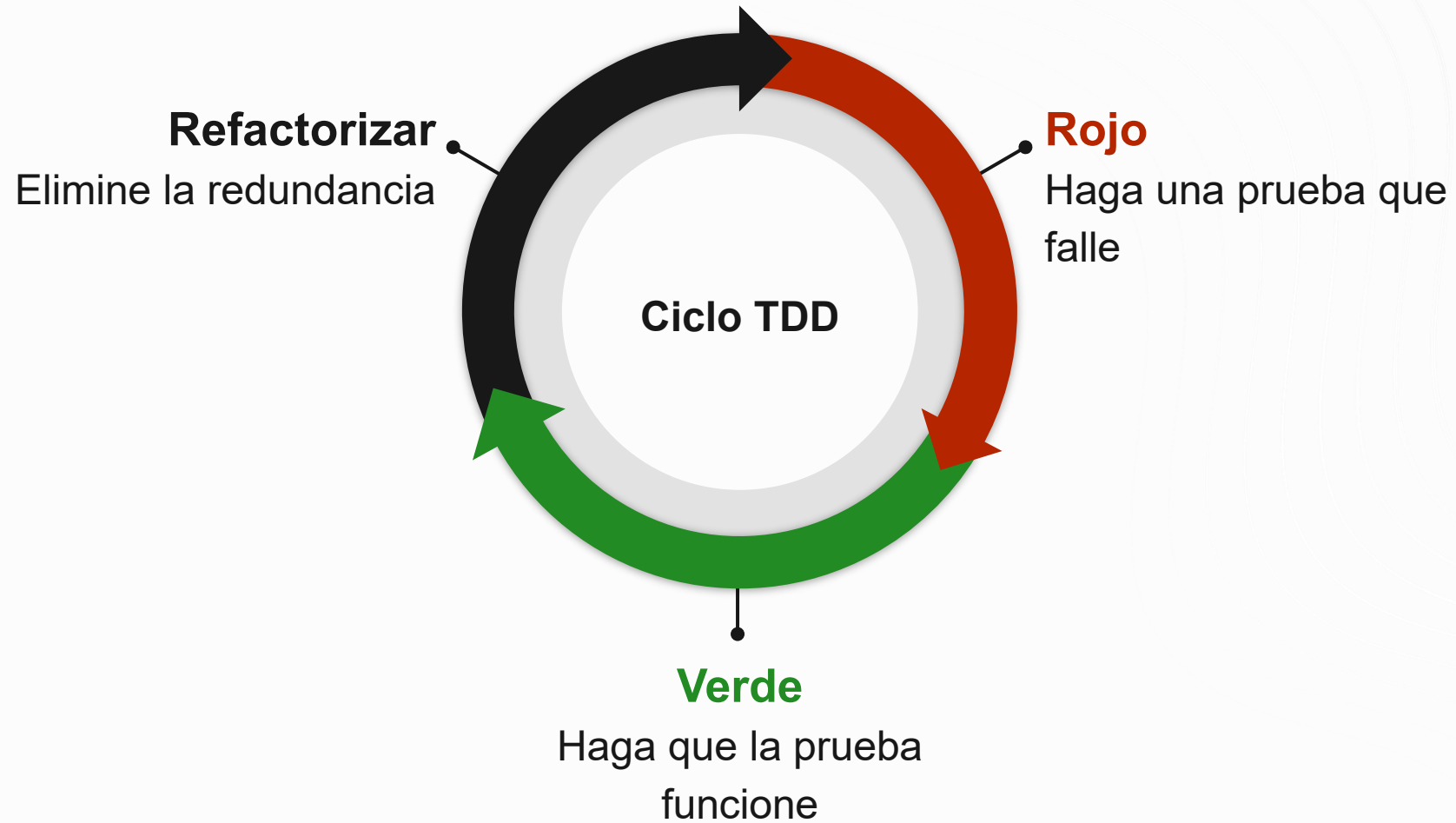
Prohibida la reproducción, total o parcial, sin autorización.

Ley nº 9610/98

Desarrollo orientado a pruebas (TDD)



Ciclo de vida del TDD



Objetivos e Impactos

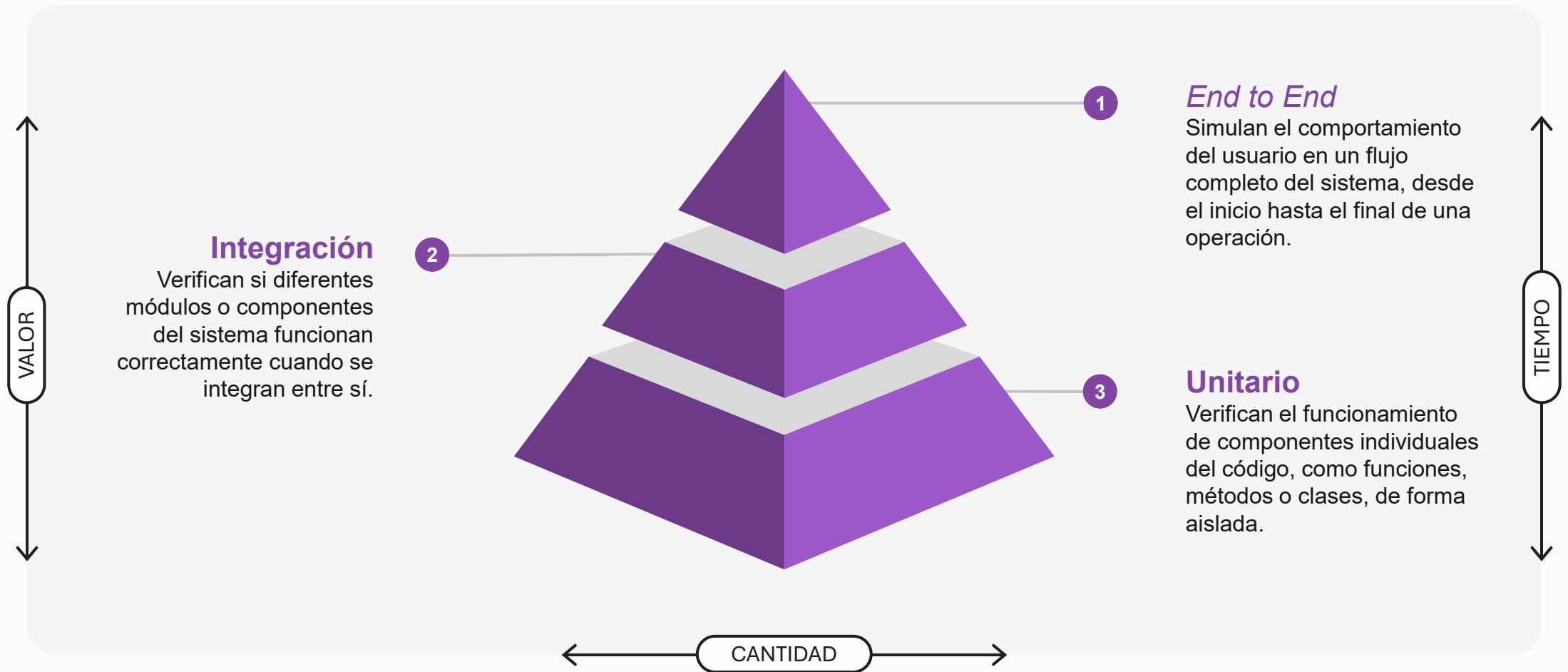
Objetivos del TDD:

- Reducir la incertidumbre y la ansiedad en el desarrollo de *software*.
- Aumentar la calidad y la confiabilidad del código desde las etapas iniciales.
- Promover un desarrollo más ágil y seguro.

Impacto del Miedo en el Trabajo:

- El miedo en exceso lleva a programadores indecisos, con menor proactividad y menos apertura a la colaboración.
- Puede afectar la motivación, causando frustración y bloqueando el aprendizaje y la creatividad.

Pirâmide de pruebas



Pruebas Unitarias

- **Qué son:** Las pruebas unitarias verifican el funcionamiento de componentes individuales del código, como funciones, métodos o clases, de forma aislada.
- **Objetivo:** Garantizar que cada unidad de código funcione correctamente de manera independiente.
- **Enfoque:** Probar una única parte del código, sin dependencia de otras partes o de sistemas externos, como bases de datos.

Ejemplo: Probar los métodos de una clase, como la validación de sus atributos.

Pruebas Unitarias

Ventajas:

- Rapidez en la ejecución, ya que prueban solo pequeñas partes del código.
- Facilidad de identificación de dónde está el problema cuando una prueba falla, ya que cada prueba se enfoca en un único componente.

Herramientas Comunes:

- JUnit (Java)
- NUnit (.NET)
- Jest (JavaScript)
- PyTest (Python)

Pruebas de Integración

- **Qué son:** Las pruebas de integración verifican si diferentes módulos o componentes del sistema funcionan correctamente cuando se integran entre sí.
- **Objetivo:** Garantizar que la interacción entre módulos de código (por ejemplo, interacción entre entidades diferentes, o entre una aplicación y su base de datos) ocurre como se espera.
- **Enfoque:** Probar el funcionamiento del sistema en conjunto, generalmente incluyendo las dependencias entre componentes.

Ejemplo: Probar la funcionalidad donde dos entidades interactúan.

Pruebas de Integración

Ventajas:

- Identificación de problemas de integración que las pruebas unitarias no pueden cubrir, como errores de comunicación entre módulos.
- Útil para verificar la compatibilidad entre diferentes partes del sistema que necesitan operar juntas.

Herramientas Comunes:

- Spring Test (Java)
- DBUnit (para probar bases de datos)
- Mockito (para simulación de dependencias).

Pruebas de *end-to-end* (E2E)

- Qué son: Las pruebas de *end-to-end* (E2E) verifican el funcionamiento de una aplicación en un flujo completo, como un todo, para asegurar que los componentes integrados proporcionan la respuesta esperada al cliente.
- Objetivo: Garantizar que el sistema funcione como un todo, validando el comportamiento del *software* desde el punto de vista del usuario final.
- **Enfoque:** Probar la aplicación entera, simulando cómo el sistema responde a operaciones reales.
- **Ejemplo:** Probar un endpoint de inicio de sesión, enviando una solicitud de autenticación de usuario y verificando si el sistema responde con éxito.

Pruebas *End-to-End* (E2E)

Vantajas:

- Confianza en que el sistema está listo para el usuario final, ya que cubre el flujo completo.
- Capacidad de identificar problemas de integración entre sistemas y flujos complejos que dependen de múltiples servicios y componentes.

Herramientas Comunes:

- Postman
- Cypress
- Swagger

Ciclo de Vida de una Prueba

- Planificación y especificación de las pruebas
- Desarrollo e implementación de las pruebas
- Ejecución y registro de los resultados
- Análisis y seguimiento de fallos



Planificación y Estrategia de Pruebas

- Definición de los criterios de aceptación y cobertura de pruebas;
- Planificación de casos de prueba y alcance de las pruebas;
- Riesgos y criterios de priorización de pruebas;
- Asignación de recursos y estimación de esfuerzo.

Buenas Prácticas en TDD

- Manter pruebas independientes y atómicas
- Nomenclatura clara y descriptiva de las pruebas
- Evitar dependencias y acoplamiento excesivo en las pruebas
- Refactorizar frecuentemente para evitar "*test code smells*"

Cobertura de Pruebas y Métricas

¿Qué es la Cobertura de Pruebas?

- Mide cuánto del código se ejecuta durante la ejecución de las pruebas.
- Indicador esencial para evaluar la eficacia y la amplitud de las pruebas.
- Ayuda a identificar áreas no cubiertas que pueden contener defectos.

Métricas de Calidad para Pruebas y Análisis de Resultados

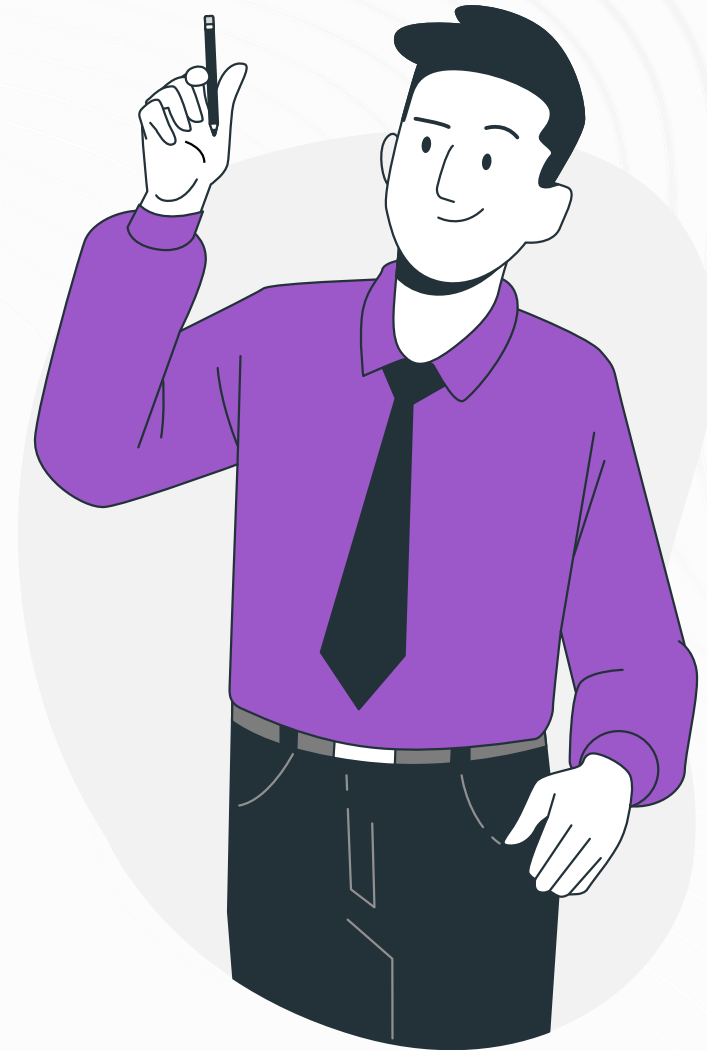
- **Tasa de Cobertura:** Porcentaje de cobertura considerado adecuado para la aplicación.
- **Métricas de Defectos:** Número y tipo de defectos encontrados en áreas cubiertas y no cubiertas.
- **Indicadores de Efectividad:** Relación entre cobertura y defectos, mostrando la precisión de las pruebas al identificar errores.
- **Tendencia a lo Largo del Tiempo:** Análisis de la evolución de la cobertura y calidad de las pruebas en cada *release*.

Herramientas para el Análisis de Cobertura de Código

- **Jacoco** (Java), **Istanbul** (JavaScript), **Coverage.py** (Python)
- **SonarQube**: Para análisis continuo de la calidad del código e integración con herramientas de CI/CD
- **Codecov y Coveralls**: Para visualización y monitoreo de cobertura de código en proyectos de código abierto y CI/CD

Gestión de Defectos

- Proceso de identificación y categorización de defectos
- Ciclo de vida de un defecto:
Detección, análisis, corrección y verificación
- Herramientas de gestión de defectos
(ej.: Jira, Bugzilla)
- Relación entre defectos y la cobertura de pruebas



Pruebas como Documentación del Sistema

Pruebas como Fuente de Documentación Viva

- Las pruebas, especialmente las automatizadas, ofrecen una visión clara de cómo el sistema debe comportarse en diferentes escenarios.
- Sirven como documentación que se actualiza automáticamente: cada vez que se modifica el código, las pruebas reflejan esos cambios.

Ventajas de las Pruebas como Documentación

- **Especificación Ejecutable:**

Las pruebas muestran cómo cada funcionalidad debe operar, sustituyendo descripciones manuales.

- **Actualización Continua:**

Cada ejecución de prueba verifica y mantiene la documentación funcional del sistema.

- **Entendimiento del Comportamiento del Sistema:**

Las pruebas unitarias, de integración y *end-to-end* explican la lógica e interacciones de los componentes en situaciones reales.

Ejemplo del ciclo TDD

```
1  import pytest
2  from uuid import uuid4
3
4  class TestUser:
5
6      # Teste para inicialização do usuário
7      def test_user_initialization(self):
8          user_id = uuid4()
9          user_name = "Test User"
10         user = User(id=user_id, name=user_name)
11
12         assert user.id == user_id
13         assert user.name == user_name
14
```



Ejemplo del ciclo TDD

```
1  from uuid import UUID
2
3  class User:
4      |
5      |     id: UUID
6      |     name: str
7      |
8      |     def __init__(self, id: UUID, name: str):
9      |         |
10         |         self.id = id
10         |         self.name = name
```

Ejemplo del ciclo TDD

```
1  import pytest
2  from uuid import uuid4
3  from domain.user.user_entity import User
4
5
6  class TestUser:
7
8      # Teste para inicialização do usuário
9      def test_user_initialization(self):
10         user_id = uuid4()
11         user_name = "Test User"
12         user = User(id=user_id, name=user_name)
13
14         assert user.id == user_id
15         assert user.name == user_name
16
```



Ejemplo del ciclo TDD

```
1 import pytest
2 from uuid import uuid4
3 from domain.user.user_entity import User
4
5 class TestUser:
6
7     # Teste para inicialização do usuário
8     def test_user_initialization(self):
9         user_id = uuid4()
10        user_name = "Test User"
11        user = User(id=user_id, name=user_name)
12
13        assert user.id == user_id
14        assert user.name == user_name
15
16    # Teste para validação do ID do usuário
17    def test_user_id_validation(self):
18        with pytest.raises(Exception, match="id must be an UUID"):
19            User(id="invalid_id", name="Test User")
20
21    # Teste para validação do nome do usuário
22    def test_user_name_validation(self):
23        user_id = uuid4()
24        with pytest.raises(Exception, match="name is required"):
25            User(id=user_id, name="")
```



Ejemplo del ciclo TDD

```
1  from uuid import UUID
2
3  class User:
4
5      id: UUID
6      name: str
7
8      def __init__(self, id: UUID, name: str):
9          self.id = id
10         self.name = name
11         self.validate()
12
13     def validate(self):
14         if not isinstance(self.id, UUID):
15             raise Exception("id must be an UUID")
16
17         if not isinstance(self.name, str) or len(self.name) == 0:
18             raise Exception("name is required")
```

Ejemplo del ciclo TDD

```
1 import pytest
2 from uuid import uuid4
3 from domain.user.user_entity import User
4
5 class TestUser:
6
7     # Teste para inicialização do usuário
8     def test_user_initialization(self):
9         user_id = uuid4()
10        user_name = "Test User"
11        user = User(id=user_id, name=user_name)
12
13        assert user.id == user_id
14        assert user.name == user_name
15
16    # Teste para validação do ID do usuário
17    def test_user_id_validation(self):
18        with pytest.raises(Exception, match="id must be an UUID"):
19            User(id="invalid_id", name="Test User")
20
21    # Teste para validação do nome do usuário
22    def test_user_name_validation(self):
23        user_id = uuid4()
24        with pytest.raises(Exception, match="name is required"):
25            User(id=user_id, name="")
```



¡Gracias!

Prof. Helder Prado Santos | <https://www.linkedin.com/in/helderprado>

MBA **USP**
ESALQ